PROJECT REPORT

Claude Code for Data Analysis

Project Report: From Installation to a Professional Trading Desk

Workspace: C:\Users\abhis\claude-analytics | Windows 11 + PowerShell

Created by

Abhishek Shinde

29 October 2026

Built with Claude Code (agentic AI coding assistant)

Contents

#	Section
1	Project overview
2	Tools, workspace and files produced
3	Module 1: Environment setup and installation
4	Module 2: Account, API key and billing setup (with troubleshooting log)
5	Module 3: Agentic data engineering (messy data, cleaning, dashboard)
6	Module 4: Data engineering pipeline (triple join)
7	Module 5: The executive audit (PDF report)
8	Module 6: Billing and usage monitoring
9	Module 7: Future Wealth forecaster
10	Module 8: Professional Trading Desk (RSI signal engine)
11	Prompt library
12	Command reference
13	Lessons learned and best practices

1. Project overview

This report documents a hands-on project in which I used **Claude Code**, an agentic AI coding assistant that runs in the terminal, to take raw business data all the way to finished analytical products: cleaned datasets, an interactive browser dashboard, a consolidated master dataset, a five-page CFO financial audit PDF, a Future Wealth investment forecaster and a trading-desk style dashboard with RSI buy/caution signals.

The project followed a course on using Claude Code for data analysis, automation and AI-powered development. Rather than writing every line of Python by hand, I described the outcome I wanted in plain English, and Claude Code planned the work, wrote and ran the scripts, checked the results and reported back. My role was to set up the environment, write clear prompts, approve changes, review the output critically and ask for corrections.

Objectives

- Install and configure Claude Code on Windows with PowerShell.
- Set up pay-as-you-go API billing and a dedicated, trusted project workspace.
- Generate realistic messy data, then clean and standardise it automatically.
- Join several financial datasets into a single master file for analysis.
- Produce executive-ready reporting: dashboards and a narrative PDF audit with charts.
- Extend the tooling with forecasting (compound growth) and technical signals (RSI).
- Monitor token usage and cost while working.

What is agentic AI?

A normal chatbot answers a question. An **agent** is given a goal and tools, and works through the steps itself: it reads files, writes code, runs it, inspects errors, fixes them and repeats until the task is done. Claude Code is that kind of agent for a project folder. It can read, edit and execute files in the folder you trust, asking permission before changes unless you allow edits for the session.

2. Tools, workspace and files produced

Tool	Purpose in this project
Windows PowerShell	Terminal used to install, configure and launch Claude Code.
Claude Code v2.1.312	Agentic coding assistant; model shown at launch: Opus 5.5, API Usage Billing.
Claude Console	Account, API key creation, $5 credit top-up and spend monitoring.
Python + pandas	Data cleaning, joining and analysis (scripts written by Claude Code).
tabulate	Prints data as formatted ASCII tables in the terminal.
FPDF2, Matplotlib/Seaborn	PDF report generation and charts embedded in the audit.
HTML / CSS / JavaScript	Interactive dashboards opened in the browser.
Node.js, Git (prerequisites)	Listed as course prerequisites for tooling and version control.

Workspace

All work happened in one dedicated folder, so Claude Code only had access to project files and not my entire user profile:

PROJECT FOLDER

```
C:\Users\abhis\claude-analytics
```

Files produced

File	Created in	Description
bank_statement.csv	Module 3	50 rows of deliberately messy bank data
clean_bank_data.py	Module 3	Reusable cleaning script: python clean_bank_data.py [input] [output]
cleaned_data.csv	Module 3	43 clean rows, ISO dates, no duplicates
dashboard.html	Module 3, 7, 8	Interactive browser dashboard, later extended
ledger_2026.csv, budget_targets.csv, marketing_roas.csv	Module 4	Source data for the pipeline
build_master_dataset.py	Module 4	Automated triple-join script
master_dataset.csv	Module 4	Consolidated ledger + budget + marketing file
financial_audit_2026.pdf	Module 5	Five-page CFO audit with narrative and charts
wealth_tracker.py	Module 7, 8	Portfolio script with forecaster and signal engine

3. Module 1: Environment setup and installation

Step 1. Install the prerequisites

Node.js, Python (with *Add Python to PATH* ticked) and Git for Windows were installed with default settings.

> **Why:** Python runs the data scripts Claude Code writes; ticking Add to PATH lets any terminal find `python`. Git gives version control and change tracking.

Step 2. Install Claude Code from PowerShell

COMMAND

```
irm https://claude.ai/install.ps1 | iex
```

> **Why:** `irm` (Invoke-RestMethod) downloads the official install script and `iex` (Invoke-Expression) runs it. The installer reported success: version 2.1.312, installed to `C:\Users\abhis\.local\bin\claude.exe`.

Step 3. Find where Claude Code is installed

COMMANDS

```
Get-Command claude
(Get-Command claude).Source
where.exe claude
```

> **Why:** These confirm which `claude.exe` PowerShell will run. `where.exe` lists every copy on the PATH, useful if more than one is installed.

Step 4. Make sure the install folder is on the PATH

COMMANDS (ONLY IF CLAUDE IS NOT RECOGNISED)

```
$env:Path -split ';' | Select-String '.local\\bin'

[Environment]::SetEnvironmentVariable("Path", $env:Path + ";$env:USERPROFILE\.local\bin", "User"
  )
```

> **Why:** The PATH tells Windows where to look for programs. The course alternative is the GUI: View Advanced System Settings > Environment Variables > Path > Edit > New, then paste the folder.

Step 5. Restart and verify

COMMANDS (IN A NEW POWERSHELL WINDOW)

```
claude --help
claude --version
```

> **Why:** PATH changes only apply to newly opened terminals, so all PowerShell windows were closed first.

4. Module 2: Account, API key and billing setup

Step 1. Create a dedicated, trusted workspace

On the first launch from the home folder, Claude Code showed an *Accessing workspace* safety check for `C:\Users\abhis`. I chose **No, exit** and created a project folder instead:

COMMANDS

```
mkdir claude-analytics
cd claude-analytics
claude
```

Why: Trusting a folder lets Claude Code read, edit and execute everything inside it. Limiting that to a project folder protects personal files elsewhere in the user profile.

Step 2. Choose pay-as-you-go API billing

I loaded the minimum **$5** of credits in the Claude Console. The Console dashboard then showed $5.00 organization credits, $0.00 spend this month against a $500 tier limit.

Why: Pay-as-you-go means paying only for tokens used, with no monthly subscription. Input tokens (reading files and prompts) cost less than output tokens (code and reports Claude writes).

Step 3. Create and store an API key

In the Console: **Get API key** > create a key > copy it once. Then save it as a permanent user environment variable:

COMMAND

```
[Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", "paste-your-key-here", "User")
```

CHECK (IN A NEW WINDOW), WITHOUT PRINTING THE KEY

```
if ($env:ANTHROPIC_API_KEY) { "Key is set" } else { "Key not found" }
```

Troubleshooting log

Setup did not work first time. These are the real problems I hit and how each was solved:

Problem observed	Cause	Fix
Your organization has disabled Claude subscription access for Claude Code	Claude Code was signed in with an account whose organisation blocks subscription use.	Switch to API-key billing through the Claude Console.
Same message after setting the key	The old sign-in was still active and took priority over the key.	Run `/logout`, `/exit`, restart `claude`, accept the detected API key (or choose Anthropic Console account).
Typing `claude "..."` at the > prompt	The `claude` command is only for PowerShell.	Inside Claude Code, type the request directly at the > prompt.
API key visible in a screenshot	The full key was pasted into a command and shared.	Revoke the key in the Console, create a new one, re-run the save command, never share it.
Remote managed settings failed to load (403)	Optional organisation policy could not be fetched.	Harmless warning: Claude Code runs with default settings. Ignore it.

Security lesson: An API key works like a password for your credit balance. Anyone holding it can spend your credits, so it belongs only in the environment variable, never in chats, screenshots or shared files.

5. Module 3: Agentic data engineering

Goal: simulate a real-world problem (messy bank exports), clean it automatically, and present it as a dashboard.

Step 1. Generate a raw, messy dataset

PROMPT USED IN CLAUDE CODE

```
Generate 50 rows of messy bank data where we will have duplicate records and messy date form
  ats... save as bank_statement.csv
```

When asked to create the file I approved it and selected **Yes, allow all edits** for the session.

> **Why:** Testing a cleaning process needs realistic problems. Generating them on purpose means every issue is known in advance, so the cleaning result can be checked.

Step 2. Clean and standardise

PROMPT USED IN CLAUDE CODE

```
Standardize all dates to YYYY-MM-DD, fill in any missing amounts using the average value, re
  move duplicates, and save the final output as cleaned_data.csv.
```

Claude Code wrote `clean_bank_data.py`, ran it, found and fixed issues in its own code, then verified each date against the original. Results it reported:

Check	Result
Rows	50 in, 43 out (7 duplicate rows removed)
Duplicates	5 exact copies; 1 differing only in spacing/capitalisation; 1 with the same date written another way. Two monthly salary payments were correctly kept as separate transactions.
Dates	All converted to YYYY-MM-DD and in order (2026-01-02 to 2026-02-13). Times, weekday names and suffixes such as 20th were handled. Ambiguous day/month dates resolved by closeness to the previous row.
Amounts	Converted to plain numbers; bracketed values like ($89.99) became negatives; currency signs and thousands separators removed.
Missing amounts	None found, so nothing imputed. The average fill step remains in the script for future files.
Text	Extra spaces trimmed; currency 'usd' capitalised to USD.
Left for my decision	TXN1018 (Apple Store) has no date; TXN1013 (Target) has no currency.

> **Key insight:** Claude Code also flagged that the average (11.91) would be misleading because it mixes income and spending, and it declined to guess the missing date. Reviewing these judgement calls is the analyst's job.

Step 3. Find where files are saved

Claude Code saves files in the folder it was started from (shown under the version on the welcome screen).

POWERSHELL

```
explorer C:\Users\abhis\claude-analytics
dir C:\Users\abhis\claude-analytics
```

INSIDE CLAUDE CODE (THE ! RUNS A SHELL COMMAND)

```
!dir
```

Step 4. Build an interactive web dashboard

PROMPT USED IN CLAUDE CODE

```
Create an interactive HTML table to preview all the data in cleaned_data.csv so it can be op
  ened as a dashboard in the browser.
```

The result, `dashboard.html`, opened in the default browser with category filters, ascending/descending column sorting, row counts, total spend and average transaction metrics.

> **Why:** A single HTML file needs no server or software install, so anyone can open and explore the data.

6. Module 4: Data engineering pipeline

Step 1. Prepare the source files

File	Contents
ledger_2026.csv	Raw general-ledger transactions (outgoing spend)
budget_targets.csv	Approved monthly spending targets per department
marketing_roas.csv	Marketing performance: ad spend mapped to attributed revenue

Step 2. Run the automated triple join

PROMPT USED IN CLAUDE CODE

```
Claude, I need to prepare a master dataset for our CFO. Join the ledger with the budget targ
  ets so we can see our variance and then map in the marketing performance data.
```

Claude proposed an automated, re-runnable script; I approved it with **Yes, allow all edits during this session**. Output: `master_dataset.csv`.

> **Why:** A script instead of one-off manual steps means next month's data can be processed by running one file. Variance (actual minus budget) is only possible once ledger and budget are joined on department.

Step 3. Preview the result in the terminal

PROMPT USED IN CLAUDE CODE

```
Claude, do not summarize. Use Python to print the first 10 rows of the master data as a form
  atted ASCII table using the tabulate library.
```

Claude offered to install the missing `tabulate` library, which I approved. **Ctrl + O** expands the collapsed output to show the full table.

> **Why:** Saying *do not summarize* forces raw rows, which is how you spot-check that a join did not drop or duplicate records.

7. Module 5: The executive audit

Step 1. Generate the first PDF draft

PROMPT USED IN CLAUDE CODE

```
Claude, I have the master_dataset.csv. I need you to write a Python script using FPDF to gen
  erate a professional PDF financial audit. Start with a one-page summary for the CFO. Highl
  ight any department that is more than 10% over budget in red. Also create a dedicated sect
  ion for marketing with campaign name, ROAS, and ad spend. Give me a list of top 5 most exp
  ensive transactions. Keep design clean, no corporate fluff.
```

Step 2. Refine the executive summary

The first draft lacked a written summary, so I asked for it explicitly:

PROMPT USED IN CLAUDE CODE

```
I do not see the one-page summary in the report that I asked for. Please include a one-page
  summary and combine the report you just created with the summary. I want the summary in te
  xt.
```

> **Why:** Agents can miss part of a request. Reviewing the draft and pointing to exactly what is missing is faster than starting again.

Step 3. Add visual analytics

PROMPT USED IN CLAUDE CODE

```
Claude, please include several charts, different from the table chart that you have used. I
  want to see a combination of bar charts and line charts that explain the data. The purpose
   of the report is to have things summarized and concise.
```

Final report structure (`financial_audit_2026.pdf`, 5 pages):

Page	Content
1	Executive summary: overall position, risk alerts, marketing return, largest transactions, audit limits and decisions needed
2	Budget vs actual: share-of-budget bar chart, monthly spend chart, department table
3	Marketing performance: KPI tiles, spend vs budget line chart, ROAS by channel
4	Campaign detail: all 36 campaigns sorted by ROAS
5	Largest transactions, Azure spend trend and method notes

Key findings from the audit

Metric	Result
Total spend vs budget	$4,391,000 against $7,728,000 (57%); $3,337,000 unspent
Departments within budget	9 of 10
Over budget (>10%, red)	Marketing: $866,000 vs $660,000, +$206,000 (+31.2%), over in 9 of 12 months
Marketing return	$4.61M attributed revenue; blended ROAS 5.32x
Best / worst channel	Mailchimp 9.7x, LinkedIn Ads 9.0x / TikTok Ads 3.5x
Best / worst campaign	Email Black Friday 11.0x / TikTok Back to School 3.1x
Top 5 transactions	All Engineering payments to Azure ($68K-$85K), $373,000 total
Azure trend	$45K to $85K per month (+89%); 66% of Engineering spend

Metric	Result
Data caveats	48 transactions dated after the report date; Sales, Customer Success and Product under 35% of budget, possibly missing spend

Decisions put to the CFO: reset or cap the Marketing budget; confirm the status of future-dated transactions; confirm the ledger is complete for low-spend departments.

8. Module 6: Billing and usage monitoring

Because API billing charges per token, I learned to monitor cost while working.

Command / place	What it shows
/cost (inside Claude Code)	Dollar cost of the current session
/status (inside Claude Code)	Account, authentication method and model in use
/model (inside Claude Code)	Switch model, e.g. to Sonnet 5.5 or Haiku 5.5 for simpler, cheaper tasks
Claude Console usage page	Monthly spend, credit balance and token volume

The course notes also reference `claude billing` and `claude usage` commands. If a version of Claude Code does not recognise them, `/cost` and the Console usage page give the same information.

> **Token dynamics:** Every edit makes Claude re-read the conversation and relevant files (context accumulation), so long sessions grow more expensive. Starting a fresh session for a new task, and using a lighter model for simple jobs, keeps costs down.

9. Module 7: Future Wealth forecaster

Next I extended the portfolio script with a forecasting feature.

PROMPT USED IN CLAUDE CODE

```
I want to add a Forecaster to our script. Add a feature where a user can enter either an ini
  tial investment (lump sum) or a monthly contribution (DCA). Using a conservative 10% annua
  l growth rate for stocks and 30% for crypto, calculate the projected value after 12 months
  . Update dashboard.html to show a "Future Wealth" section with the stock and crypto projec
  tions side by side.
```

How the projection works

Lump sum: the whole amount grows for one year: future value = P x (1 + g). At 10%, $1,000 becomes $1,100; at 30%, $1,300.

Monthly contribution (DCA, dollar-cost averaging): each deposit grows only for the months remaining. With a monthly rate r = $(1 + g)^{1/12} - 1$, the value after 12 deposits of C is C x $((1 + r)^{12} - 1) / r$.

> **Caveat:** The growth rates are fixed assumptions for illustration, not predictions. Real returns vary, and crypto in particular can fall sharply, so the dashboard shows projections, not guarantees.

10. Module 8: Professional Trading Desk

The final step upgraded the tool into a trading-desk style dashboard with technical signals.

PROMPT USED IN CLAUDE CODE

```
We are upgrading our tool into a professional Trading Desk. First read @wealth_tracker.py an
  d @dashboard.html; this is our current foundation.

1. Refactor the Python script and add a "Signal" engine that calculates a 14-day RSI (Relati
  ve Strength Index) for all 20 assets.
2. Update dashboard.html with a Bloomberg-style dark mode (CSS).
3. If an asset's RSI is below 30, color its row neon green with a "BUY" tag.
4. If an asset's RSI is above 70, color its row red with a "CAUTION" tag.
5. Keep all existing "Future Value" logic exactly as it is.
```

> **Why the prompt is written this way:** The `@filename` syntax points Claude Code at specific files so it reads the current code before changing it. Saying *keep the Future Value logic exactly as it is* protects earlier work, since Claude may not remember previous sessions.

What RSI measures

RSI compares the size of recent gains with recent losses over 14 days and scores the result from 0 to 100: RSI = 100 - 100 / (1 + average gain / average loss).

RSI range	Reading	Dashboard treatment
Below 30	Oversold: price has fallen fast	Neon green row, BUY tag
30 to 70	Neutral	Standard row
Above 70	Overbought: price has risen fast	Red row, CAUTION tag

RSI is one indicator among many and these tags are educational signals, not investment advice.

11. Prompt library

All prompts used in the project, in order, for reuse.

1. GENERATE MESSY DATA

```
Generate 50 rows of messy bank data where we will have duplicate records and messy date form
  ats... save as bank_statement.csv
```

2. CLEAN DATA

```
Standardize all dates to YYYY-MM-DD, fill in any missing amounts using the average value, re
  move duplicates, and save the final output as cleaned_data.csv.
```

3. DASHBOARD

```
Create an interactive HTML table to preview all the data in cleaned_data.csv so it can be op
  ened as a dashboard in the browser.
```

4. TRIPLE JOIN

```
Claude, I need to prepare a master dataset for our CFO. Join the ledger with the budget targ
  ets so we can see our variance and then map in the marketing performance data.
```

5. TERMINAL PREVIEW

```
Claude, do not summarize. Use Python to print the first 10 rows of the master data as a form
  atted ASCII table using the tabulate library.
```

6. PDF AUDIT

```
Claude, I have the master_dataset.csv. I need you to write a Python script using FPDF to gen
  erate a professional PDF financial audit. Start with a one-page summary for the CFO. Highl
  ight any department that is more than 10% over budget in red. Also create a dedicated sect
  ion for marketing with campaign name, ROAS, and ad spend. Give me a list of top 5 most exp
  ensive transactions. Keep design clean, no corporate fluff.
```

7. AUDIT SUMMARY FIX

```
I do not see the one-page summary in the report that I asked for. Please include a one-page
  summary and combine the report you just created with the summary. I want the summary in te
  xt.
```

8. AUDIT CHARTS

```
Claude, please include several charts, different from the table chart that you have used. I
  want to see a combination of bar charts and line charts that explain the data. The purpose
   of the report is to have things summarized and concise.
```

9. FORECASTER

```
I want to add a Forecaster to our script. Add a feature where a user can enter either an ini
  tial investment (lump sum) or a monthly contribution (DCA). Using a conservative 10% annua
  l growth rate for stocks and 30% for crypto, calculate the projected value after 12 months
  . Update dashboard.html to show a "Future Wealth" section with the stock and crypto projec
  tions side by side.
```

10. TRADING DESK

```
We are upgrading our tool into a professional Trading Desk. First read @wealth_tracker.py an
  d @dashboard.html; this is our current foundation. Refactor the Python script and add a "S
  ignal" engine that calculates a 14-day RSI for all 20 assets. Update dashboard.html with a
   Bloomberg-style dark mode (CSS). RSI below 30: neon green row with a "BUY" tag. RSI above
   70: red row with a "CAUTION" tag. Keep all existing "Future Value" logic exactly as it is
  .
```

12. Command reference

Command	Where	Purpose
irm https://claude.ai/install.ps1 | iex	PowerShell	Install Claude Code
Get-Command claude / where.exe claude	PowerShell	Show the install location
claude --help, claude --version	PowerShell	Verify the install
mkdir claude-analytics; cd claude-analytics	PowerShell	Create and enter the workspace
claude	PowerShell	Start Claude Code in the current folder
[Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", ..., "User")	PowerShell	Save the API key permanently
if ($env:ANTHROPIC_API_KEY) {...}	PowerShell	Check the key without printing it
explorer / dir <folder>	PowerShell	Open or list project files
python clean_bank_data.py [input] [output]	PowerShell	Re-run the cleaning pipeline
/status	Claude Code	Account, auth method and model
/logout, /login	Claude Code	Switch account or billing method
/model	Claude Code	Choose Opus, Sonnet or Haiku
/cost	Claude Code	Session spend
/exit	Claude Code	Leave Claude Code
!<command>	Claude Code	Run a shell command, e.g. !dir
@<file>	Claude Code	Reference a file in a prompt
Ctrl + O	Claude Code	Expand collapsed output

13. Lessons learned and best practices

- **Use a dedicated workspace.** Trust only the project folder, never the whole user profile.
- **Know where commands go.** `claude` is typed in PowerShell; requests are typed at Claude Code's > prompt, and slash commands control the session.
- **Protect the API key.** Store it in an environment variable, check it without printing it, and revoke it immediately if it is ever exposed.
- **Read error messages literally.** The organisation-access message named its own fix (use an API key); the 403 settings warning was harmless.
- **Be specific in prompts.** Name file names, formats (YYYY-MM-DD), thresholds (10%, RSI 30/70) and what must not change.
- **Review and iterate.** The audit improved over three prompts; pointing to exactly what was missing worked best.
- **Ask for scripts, not one-off answers.** Re-runnable Python files turn a single analysis into a pipeline.

- **Keep judgement human.** Claude flagged ambiguous dates, a misleading average and missing values; deciding what to do with them is the analyst's call.
- **Watch the cost.** Check `/cost`, use lighter models for simple work and start fresh sessions for new tasks.

Prepared by **Abhishek Shinde**, 29 October 2026.